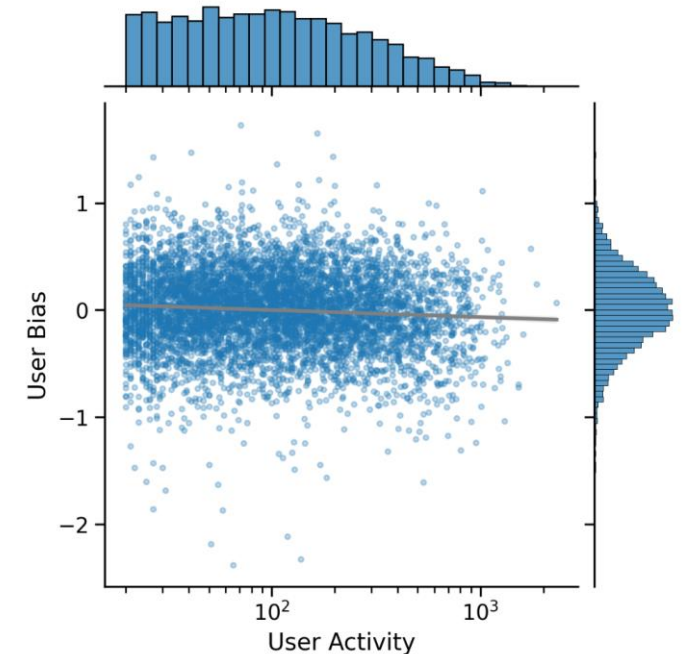
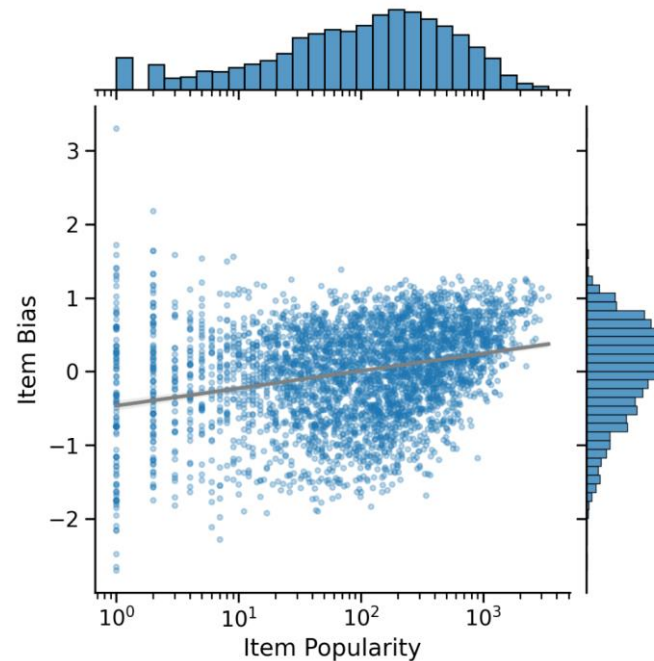
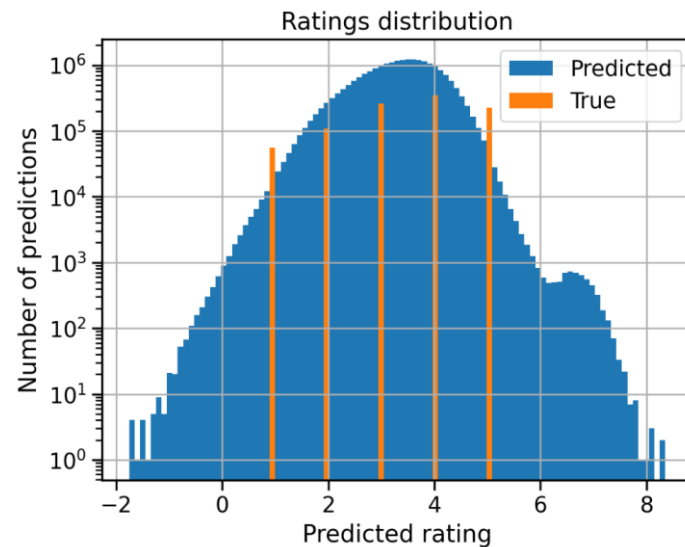


Recommender Systems

Lecture 3

Previous lecture

- simple non-personalized ranking
 - based on popularity / average ratings
 - requires adjustments to take into account trends / non-stationarity / biasedness
- baseline predictors as a solution to optimization problem

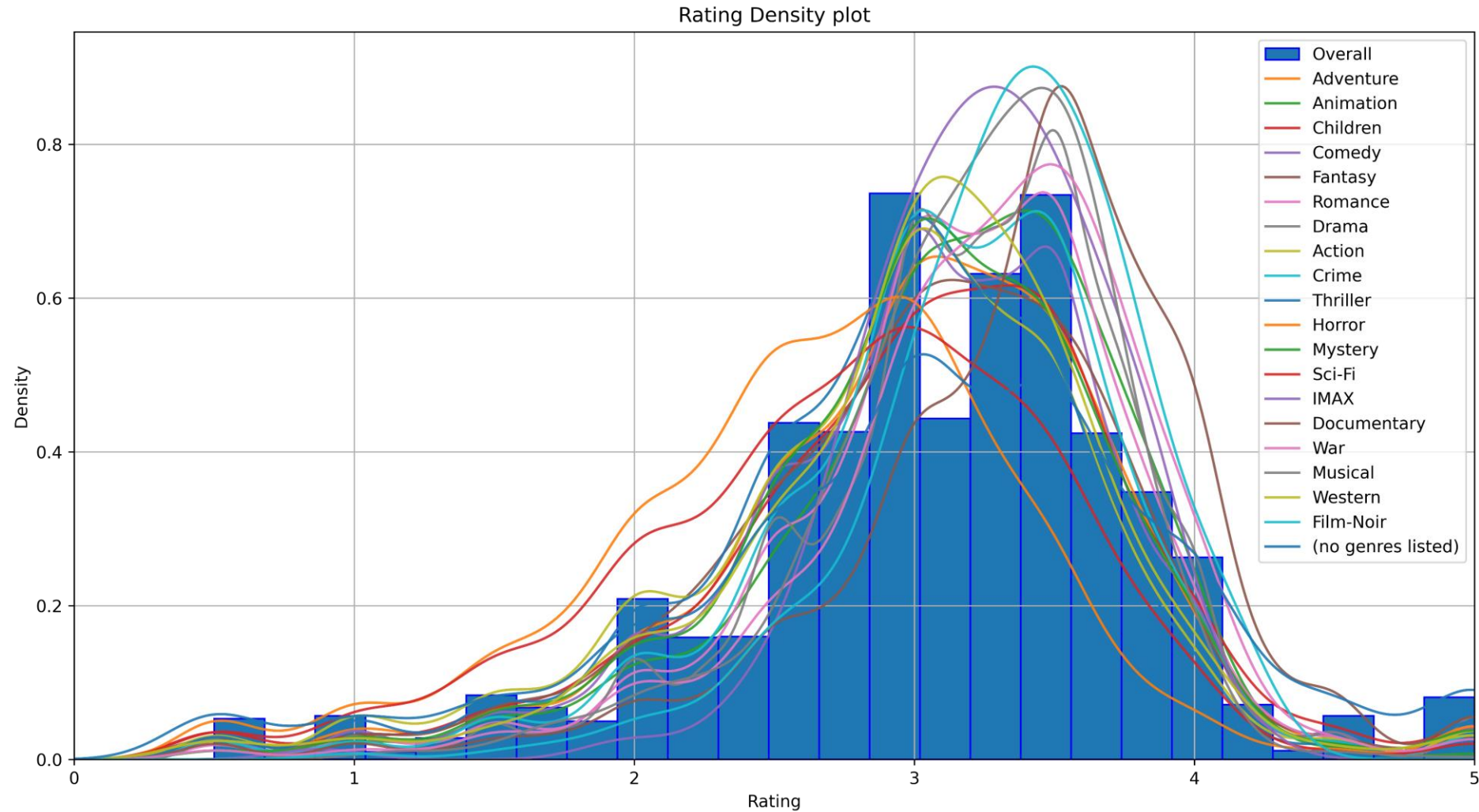


Today's Lecture

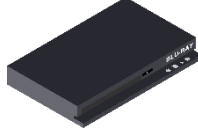
- Content-based recommendations
- Basic implementations

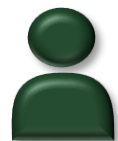
Content-based Recommendations

Ratings distribution over movie genres (ML-25M)



Using content for matching

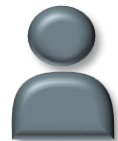
			
Good for gaming	+		+
Good for movies	+	+	+
Good for TV shows	+		
Blue-ray support		+	+



Gamer



Cinema fan



“Family guy”

generating recommendations require:

item features to compute items relevance

user profile information to match items

previous user actions to match items

user attributes for tailoring predictions

criteria specified by user / expert

- knowledge-based
- case-based
- constraint-based

Typical use case: “cold” items

Example of content features

Users

The screenshot shows a Goodreads user profile for 'Steve'. The profile includes a circular profile picture of a man reading a book, a 'Follow' button, and an 'Add friend' button. Below the profile picture, it shows '262 ratings (3.77 avg)' and '114 reviews more photos (1)'. A section titled 'STEVE'S FAVORITE BOOKS' displays three book covers. Below this, a 'Like' button is visible. A section titled 'This is my journey in books for 2021!' shows a progress bar and two book covers: 'LOVER ED' (Shortest Book, 228 pages) and 'HARUKI MURAKAMI' (Longest Book, 704 pages). At the bottom, it states 'Average book length in 2021 365 pages'.

- More attributes:
- demographics
 - location
 - occupation
 - ...

Items

The screenshot shows a Goodreads book page for 'Mindset: The New Psychology of Success' by Carol S. Dweck. The page includes a book cover, a 'Want to Read' button, and a 'Rate this book' section with five stars. Below this, there is a 'GET A COPY' section with 'Amazon' and 'Stores' buttons. The book description states: 'After decades of research, world-renowned Stanford University psychologist Carol S. Dweck, Ph.D., discovered a simple but groundbreaking idea: the power of mindset. In this brilliant book, she shows how success in school, work, sports, the arts, and almost every area of human endeavor can be dramatically influenced by how we t ...more'. A 'FRIEND REVIEWS' section prompts users to sign up to see what their friends thought. A 'READER Q&A' section prompts users to sign up to ask other readers questions. On the right, there is a 'Share' section with social media icons, a 'Recommend It | Stats | Recent Status Updates' section, a 'READERS ALSO ENJOYED' section with book covers for 'Atomic Habits', 'GRIT', and 'The Power of Habit', and a 'GENRES' section with a list of genres and user counts.

GENRES	Users
Psychology	2,453 users
Nonfiction	2,158 users
Self Help	1,469 users
Business	737 users
Self Help > Personal Development	646 users
Education	597 users
Leadership	328 users
Parenting	327 users
Productivity	177 users
Teaching	173 users

- Other features:
- price
 - format/style
 - language
 - ...

Item content for rating prediction

Task: find a utility function f :

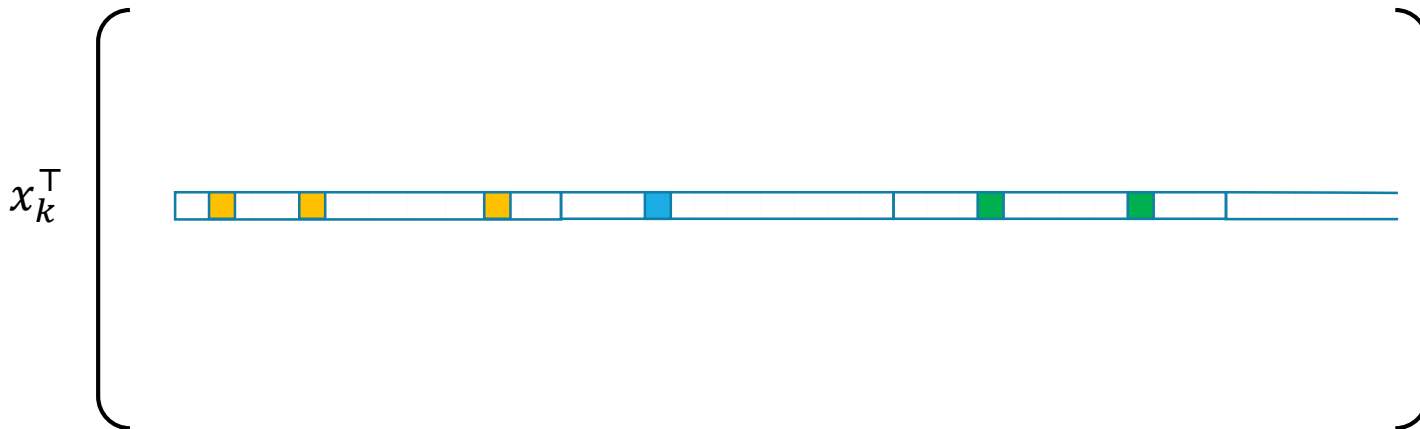
$f(\text{item features}) \rightarrow \text{feedback}$

Linear regression model: $X\mathbf{w} = \mathbf{y}$

$\mathbf{w}$ – learnable weights

$\mathbf{y}$ – target ratings

X – design matrix



Linear w.r.t. what?

Using linear regression as model for prediction

$$\mathbf{y} = X\mathbf{w} + \boldsymbol{\epsilon}$$

$$y_k = \mathbf{x}_k^\top \mathbf{w} + \epsilon_k$$

Under assumption of normally distributed noise,
standard linear least squares problem:

$$\|\mathbf{y} - X\mathbf{w}\|_2^2 \rightarrow \min$$

Multicollinearity problem

Multicollinearity problem

Typical ways to combat the ill-posedness:

- add regularization term
 - e.g., l2-penalty $\lambda \|\mathbf{w}\|^2$
- feature selection
 - hard problem*
- feature transformation
 - e.g., dimensionality reduction

* Nikitin A. et al. Are quantum computers practical yet? A case for feature selection in recommender systems using tensor networks // arXiv preprint arXiv:2205.04490. – 2022.

* Sukhorukov N., et al. Maximum Impact with Fewer Features: Efficient Feature Selection for Cold-Start Recommenders through Collaborative Importance Weighting // arXiv preprint arXiv:2508.06455. – 2025.

How to select features?

Types of features:

- Known knowns
- Known unknowns
- Unknown unknowns

Adding/omitting regressors:

- Missing important variables → bias
- Adding meaningless variables → overfitting
- No standard “silver bullet” recipe exists.
- Choose the simplest possible model, but not simpler.

Simpson Paradox

Correlation vs causation

Is watching the 1984 Ghostbusters movie killing people?



Age Distribution of UK residents Age 10-59yr
watching 1984 Ghostbusters and 2021 Ghostbusters

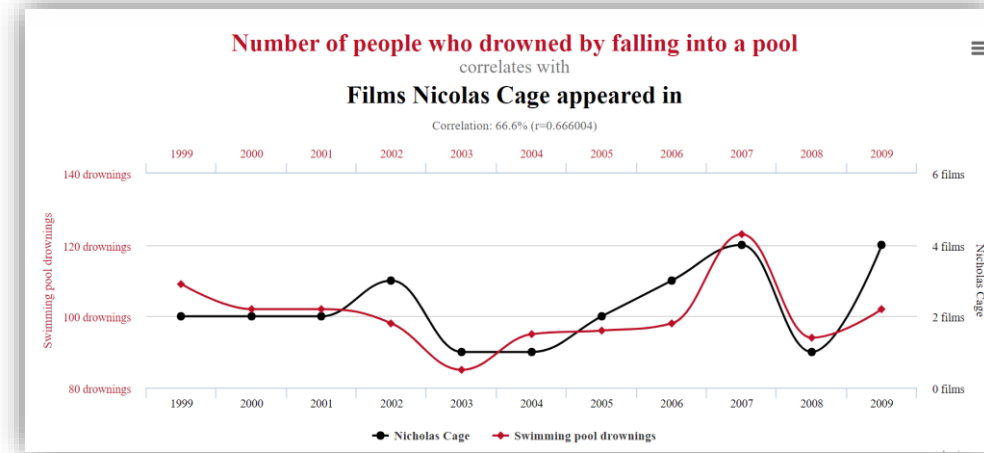
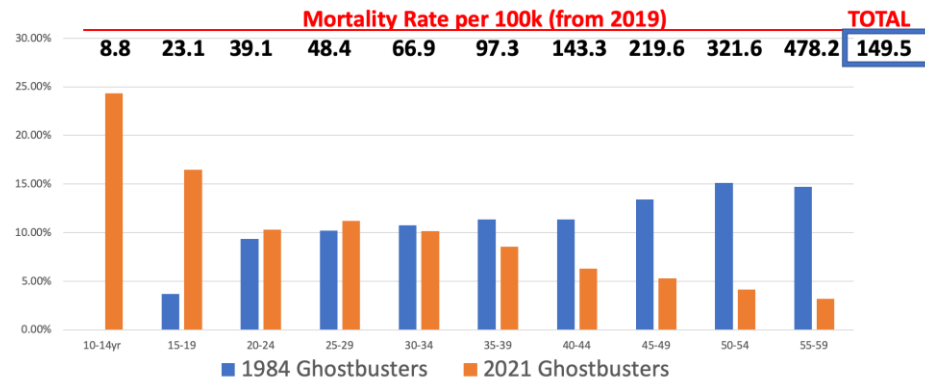


Image from: <https://tylervigen.com/spurious-correlations>

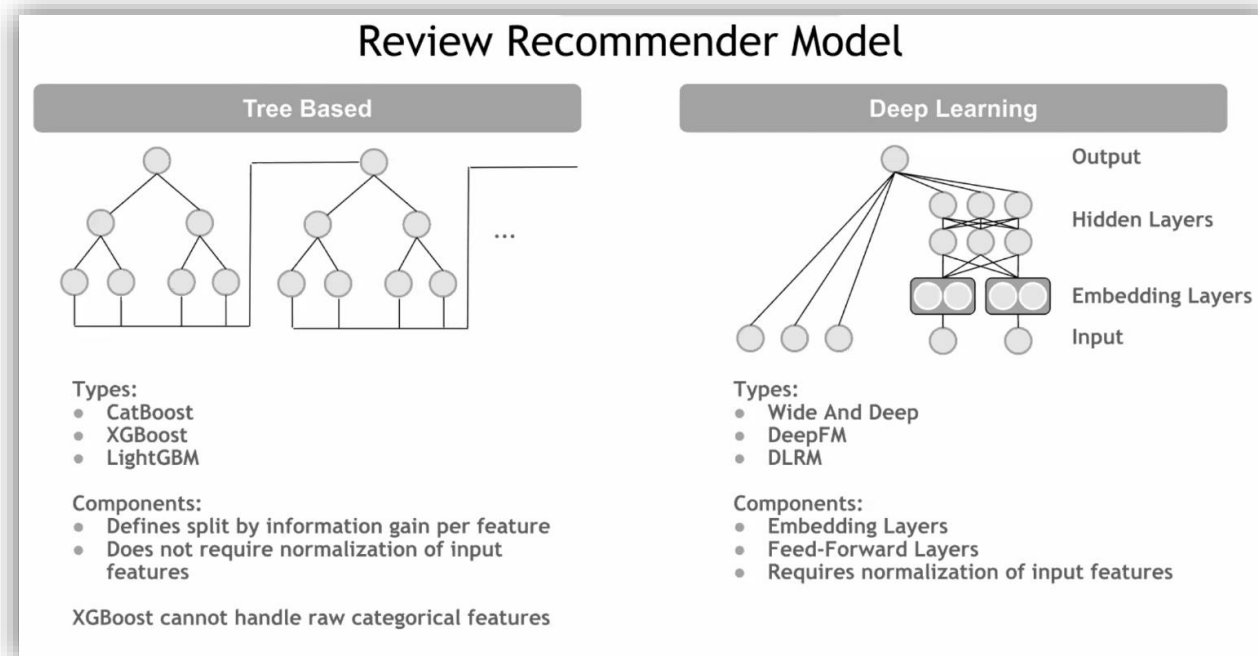
Other methods for content-based filtering

- SVM
- Decision Tree-based models
 - CatBoost
 - LightGBM
 - XGBoost
 - Automated feature selection, e.g. as in LightAutoML
- Neural Networks

Feature engineering example

Feature Type	Example	Feature Engineering
Categorical	- User ID / Item ID - Brand - Main Category	- Target Encoding - Count Encoding - Categorify + Combining Categories
Unstructured list	- Keywords - Subcategories - Colors	- Target Encoding - Count Encoding - Categorify
Numeric	- Price - Deliver time - Avg. reviews	- Binning - Normalization - Gauss Rank
Timestamp	Timestamp	Extract month, weekday, weekend, hour
Timeseries	- Events in order - Time since last event	# of events in past X - Difference in time (lag)
Image	Product image	Extract latent representation with deep learning
Text	Description	Extract latent representation with deep learning
Social graph	Follower/Following graph	Link analysis
Geo location	Addresses	Distances to point of interest

- Massive experiments in parallel
- Trying, not guessing



How to Build a Winning Deep Learning Recommender System

<https://medium.com/rapids-ai/winning-solution-of-recsys2020-challenge-gpu-accelerated-feature-engineering-and-training-for-cd67c5a87b1f>

https://www.youtube.com/watch?v=bHuww-l_Sqo

Modeling binary feedback

Task: predict whether user will like an item or not.

Example:

$$y_k = \begin{cases} 1, & \text{rating above threshold,} \\ 0, & \text{otherwise.} \end{cases}$$

In linear regression: $y_k \approx \mathbf{x}_k^\top \mathbf{w}$.

How to translate $\mathbf{x}_k^\top \mathbf{w} \in \mathbb{R}$ into binary domain?

Classification task via logistic regression

The model:

$$p_{\mathbf{w}}(y_k = 1 \mid \mathbf{x}_k) \sim \frac{1}{1 + e^{-\mathbf{x}_k^\top \mathbf{w}}}$$

Target:

$$y_k = \begin{cases} 1, & \text{if } \mathbf{x}_k^\top \mathbf{w} > 0, \\ 0, & \text{otherwise.} \end{cases}$$

- Our observations data can be viewed as a set of random trials.
- Outcome of each trial is binary (coin flip).

$$y \sim \text{Bernulli}(\mu)$$

$$E[y] = \mu, \quad 0 \leq \mu \leq 1$$

A bit of probability theory

- We aim to maximize the joint probability distribution (likelihood)

$$p_{\mathbf{w}}(\mathbf{y}, X) = \prod_{k=1}^n p_{\mathbf{w}}(y_k, \mathbf{x}_k), \quad p_{\mathbf{w}}(y_k, \mathbf{x}_k) = p_{\mathbf{w}}(y_k \mid \mathbf{x}_k) \cdot p_{\mathbf{w}}(\mathbf{x}_k)$$

$$\operatorname{argmax}_{\mathbf{w}} \prod_{k=1}^n p_{\mathbf{w}}(y_k, \mathbf{x}_k) = \operatorname{argmax}_{\mathbf{w}} \log \prod_{k=1}^n p_{\mathbf{w}}(y_k, \mathbf{x}_k)$$

$$= \operatorname{argmax}_{\mathbf{w}} \left(\log \prod_{k=1}^n p_{\mathbf{w}}(y_k \mid \mathbf{x}_k) + \log \prod_{k=1}^n p_{\mathbf{w}}(\mathbf{x}_k) \right)$$

$$= \operatorname{argmax}_{\mathbf{w}} \sum_{k=1}^n \log p_{\mathbf{w}}(y_k \mid \mathbf{x}_k)$$

Maximum likelihood estimation

$$\mathcal{L}(\mathbf{w}) = \sum_{k=1}^n \log p_{\mathbf{w}}(y_k \mid \mathbf{x}_k)$$

- We assume data distribution is independent of model parameters, i.e. $p_{\mathbf{w}}(\mathbf{x}_k) = p(\mathbf{x}_k)$ (discriminative formulation)

- The probability of each outcome is parametrized via:

$$p_{\mathbf{w}}(y_k \mid \mathbf{x}_k) = p_{\mathbf{w}}(y = 1 \mid \mathbf{x}_k)^{y_k} \cdot (1 - p_{\mathbf{w}}(y = 1 \mid \mathbf{x}_k))^{(1-y_k)}$$

- Leads to

$$\mathcal{L}(\mathbf{w}) = - \sum_{k=1}^n \log \left(1 + e^{-\mathbf{x}_k^{\top} \mathbf{w}} \right) - \sum_{k: y_k=0} \mathbf{x}_k^{\top} \mathbf{w}, \quad y_k \in \{0, 1\}$$

Logistic Regression Objective

- In a more concise and practical form:

$$\mathcal{L}(\mathbf{w}) = - \sum_{k=1}^n \log \left(1 + e^{-y_k \cdot \mathbf{x}_k^T \mathbf{w}} \right), \quad y_k \in \{-1, +1\}$$

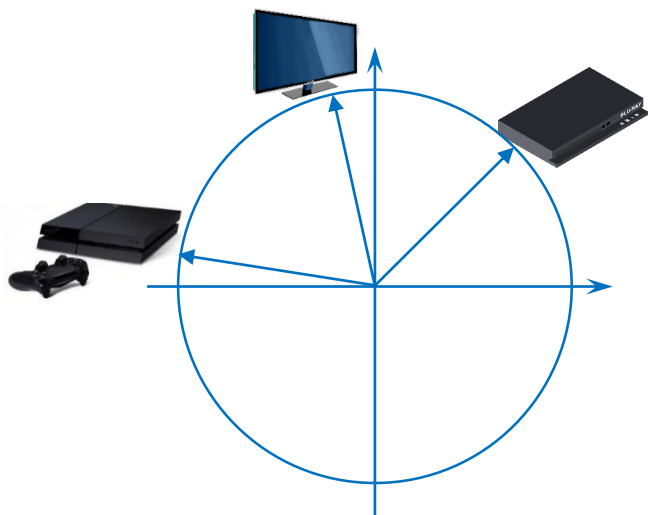
- Turns into standard loss minimization:

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} Q(\mathbf{w}), \quad Q(\mathbf{w}) = -\mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

- No closed form solution
- Can be solved with e.g.:
 - Gradient-based methods (e.g., SGD)
 - Newton-Raphson (or Iteratively Reweighted Least Squares)
 - Quasi-Newton methods (L-BFGS)

Similarity-based models

Each item as a vector in d -dimensional feature space.



Given two item feature vectors $\mathbf{x}_i$ and $\mathbf{x}_j$, we can estimate their proximity on a unit sphere:

$$\cos(\alpha) = \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\|\mathbf{x}_i\| \cdot \|\mathbf{x}_j\|}$$

What happens if some features are missing?

How to match with a user?



User profile:

$$\bar{\mathbf{x}}_u = \sum_{j \in I_u} w_{uj} \mathbf{x}_j, \quad I_u - \text{set of items of user } u$$

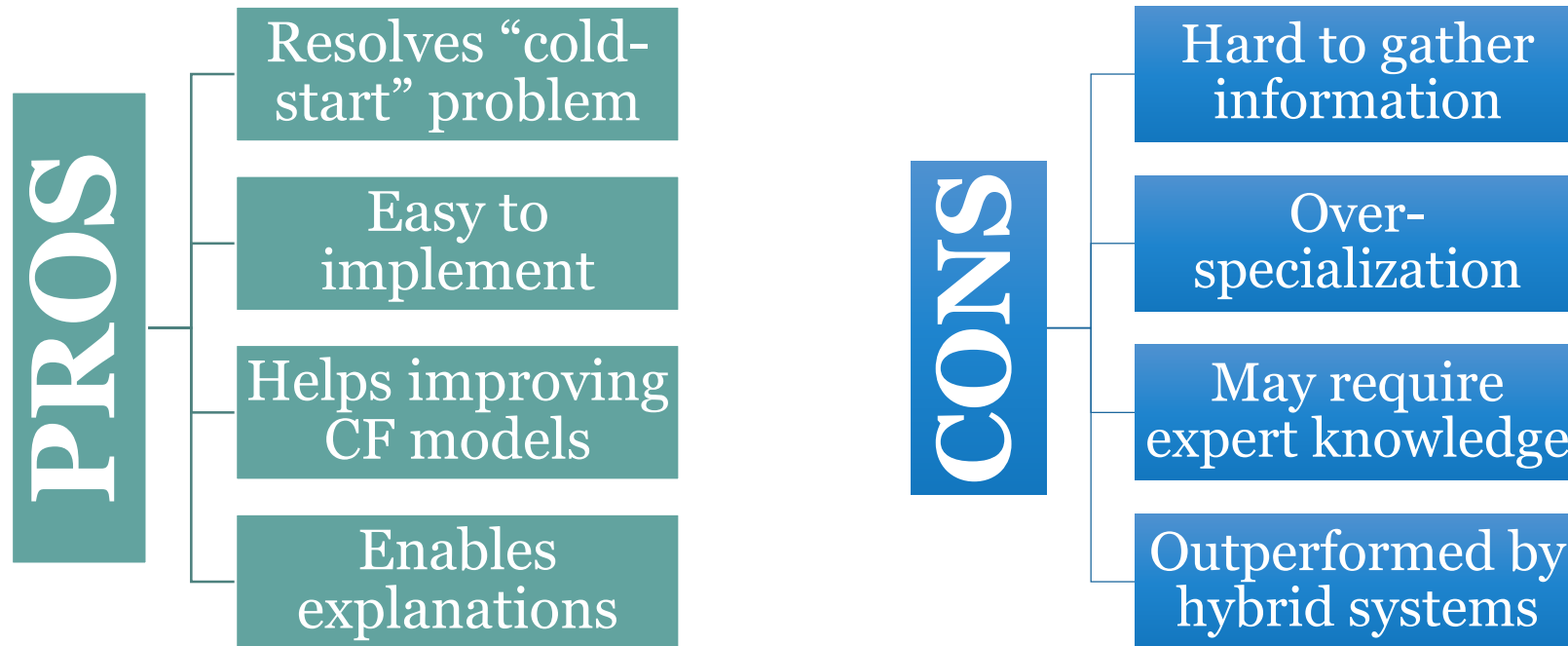
The simplest case:

$$w_{uj} = \frac{1}{|I_u|} \quad \text{or} \quad w_{uj} = \frac{r_{uj}}{\sum_k r_{uk}},$$

Relevance estimation:

$$r_{ui} = \frac{\bar{\mathbf{x}}_u^\top \mathbf{x}_i}{\|\bar{\mathbf{x}}_u\| \cdot \|\mathbf{x}_i\|}$$

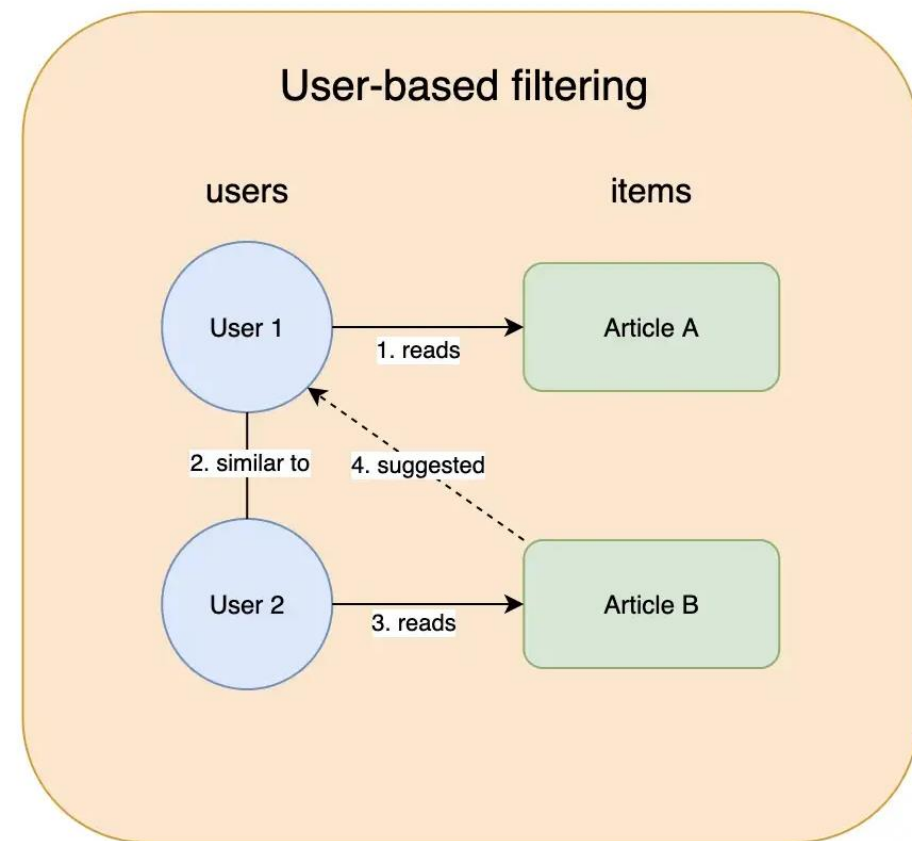
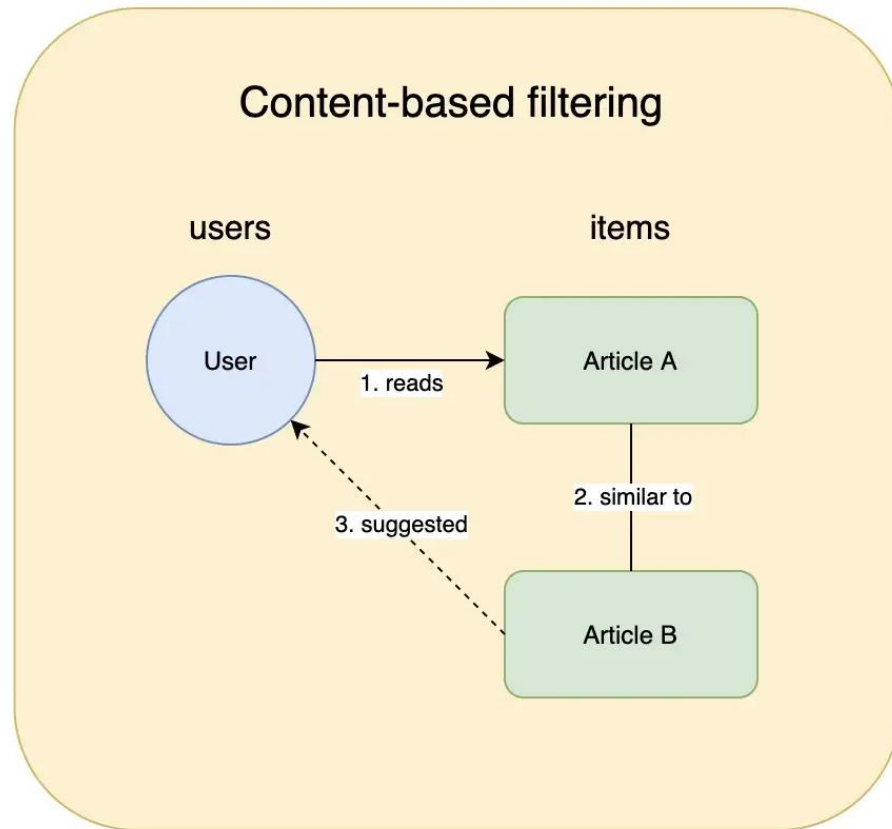
Content-based approach summary



That's where **deep learning** can really help: **features embedding**.

Better in hybrid approaches.

User-based filtering



- still content-based filtering
- relevant scenario: user is a target (“cold” user)

How to evaluate

- Intuitive heuristic based on a self-consistency check:
 - recommendations must be close to what a user likes
 - recommendations must be distant from what a user dislikes
- For example:
 1. select pairs of known likes and dislikes for each user
 - these are used for the test → must be excluded from training data
 2. identify top-ranked items from a user history
 - using true ratings (if present) or predicted relevance scores (binary feedback)
 3. compare proximity of the items to the test pairs for each user
 - proximity can be calculated based on predicted relevance scores
 - compare via e.g. aggregation of proximity values
 - alternatively:
 - build a user profile first using top-ranked items
 - compare proximity of the user profile to the user's test pairs

Practice session